Classes

⇒ template / blueprint for creating objects

Syntax:

→ use of keyword class

→ Always add a method named constructor()

```
class ClassName {
    constructor(){...}
}
```

Example

```
class Car{
    constructor(name, year){
        this.name = name;
        this.year = year;
    }
}
```

Using Class:

```
const myCar1 = new Car("Ford", 2014);
console.log(myCar1.name);
```

Constructor Method

→ It have exact name constructor

→ It is executed automatically when new object is created.

→ It used to initialize object properties

→ If constructor method is not defined, the javascript will add an empty constructor by default.

Class Method

→ created after constructor

→ defines the behavior of object of class

→ define directly in class body without using function keyword.

class

Properties:

Properties store data specific to each object created from the class.

They are set within constructor using this.propertyName = value;

Class Inheritance

→ a class can all functionality from a parent class and allows you to add more.

→ a class can inherit all methods & properties of another class.

→ `extend` keyword is used to inherit

Example:

```
Class Person {
    constructor(name){
        this.name = name;
    }
  greet() {
    console.log('Hello, ${this.name}');
  }
}
```

```
class Student extends Person{
}
let std = new Student('Jack');
std.greet();
```

super() keyword

→ The super keyword inside a child denotes its parent class.

```
class Student extends Person{
    constructor(name){
        console.log("Student Class");
        super(name);
    }
}
let std = new Student('Jack');
std.greet();
```

Overriding Method or Properties

→ If a child class has the same method or property name as that parent class

→ It will used the method & property of child class.

Prototypal Inheritence

→ Uses object.creat() or functions

→ Objects inherit directly from other objects.

→ Everything is objeat in JS.

→ Under Hood JS always uses prototypes

```
const animal = {
    speak() {
```

```
    console.log("Animal sound");
  }
};

const dog = Object.create(animals);
dog.bark = function() {
  console.log("Woof");
};

dog.speak();
dog.bark();
```

Class based Inheritence

→ Uses class, extend, super

→ objects inherits from class

→ class is syntactic sugar over prototypes.